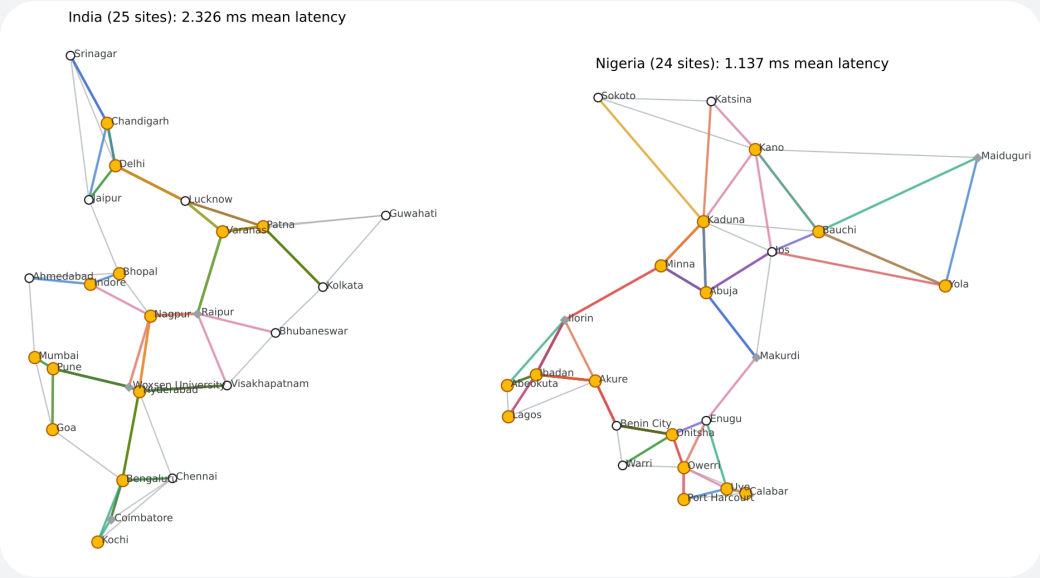
DESIGN AND ANALYSIS OF ALGORITHMS · PROJECT

Lightpath.

Latency-aware cloud task placement on network graphs

Where should copies of each cloud service run, so every user is one short path away, without overloading servers or breaking the budget?



Naman Singla · 25WU0101084
Mohak Gupta · 25WU0101074

Faculty: Dr. Monday Jubrin Abdullahi
Section: CSE RHINOS · 2026–27

THE PROBLEM

Where should each service run so every user gets a fast answer?

A user in Kochi served from Delhi waits for every request to cross the country. A copy closer by is fast, but every copy costs money and uses server capacity.

Nodes

Servers / cities

each with CPU, memory and a $/hour cost

Edges

Fibre links

weight = **latency in ms**, from real distance

Demand

Users

requests per second from each city for each service

Goal: minimise average latency from every user to the nearest copy · within **server capacity** · within **budget**

Why it is hard: it contains the classic **p-median** problem, which is **NP-hard**. So we solve the easy parts exactly, the hard part with fast greedy methods, and measure how close they get to the true optimum.

THE SOLUTION

Seven stages — each a classic algorithm chosen for what it guarantees

1

Latency matrix
Floyd–Warshall / Johnson
How far is every city from every other?

2

Backbone
Prim / Kruskal MST
Cheapest links connecting all cities

3

Routing
Dijkstra / Bellman-Ford
Which path does each request take?

4

Server activation
0/1 knapsack DP
Which servers to switch on within budget?

5

Replica sizing
Multiple-choice knapsack
How many copies of each service?

6

Placement
4 greedy schedulers
Exactly where does each copy go?

7

Refinement
Local search
Move / swap copies until nothing improves

✓ **Exact check (the referee):** on smaller networks a **mixed-integer program** (MILP, HiGHS solver) finds the **proven optimum**. Every heuristic reports its **gap** from it — so we never just claim our answer is good, we prove how good.

10 test networks

- **Real:** India, Nigeria, Woxsen campus, Abilene, NSFNET
- **Barabási–Albert:** a few big hubs, like the internet
- **Waxman:** nearby servers linked, like an ISP
- **Fat-tree:** switch layers inside a data centre
- **Erdős–Rényi:** random links, density on one knob
- Synthetic ones let us vary size and density

Why four shortest-path algorithms?

One source

Dijkstra

Settles the closest city first; its distance is then final. **Fast**, stops early at the destination.

`O((V+E) log V)`

Fails with negative weights

One source

Bellman-Ford

Relaxes every edge V−1 rounds. **Handles rebates** (negative costs) and detects negative cycles.

`O(V·E)`

Used for $/GB transfer routes

All pairs

Floyd–Warshall

DP over intermediate cities: d[i][j] = min(d[i][j], d[i][k]+d[k][j]). Builds the full latency table.

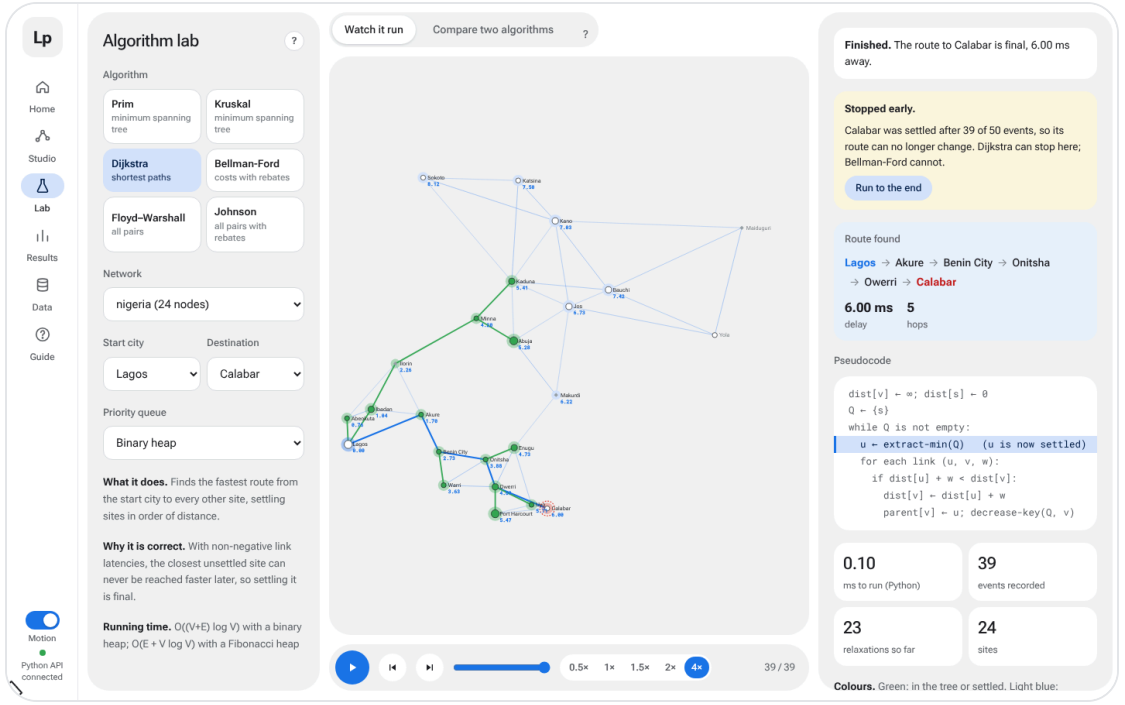
`Θ(V³)`

All pairs

Johnson

Bellman-Ford once to reweight edges ≥ 0, then Dijkstra from every city. Better on **sparse** graphs.

`O(VE + V² log V)`



Algorithm Lab: Dijkstra from Lagos to Calabar, step by step with pseudocode — the blue line is the final route. The code **picks automatically**: sparse → Johnson/Dijkstra, dense → Floyd–Warshall.

STAGES 2, 4 & 5 · SPANNING TREES AND DYNAMIC PROGRAMMING

Build the backbone, then spend the budget wisely

Minimum spanning tree

Prim vs Kruskal

Prim grows one tree using a priority queue. **Kruskal** sorts edges and uses **union-find** to skip cycles.

Both give the **same cheapest tree** (cut property). Rule: Prim if **E > V log V** (dense), else Kruskal.

`O(E log V) · O(E log E)`

Measured: sparse → Kruskal 1.5 ms vs Prim 3.7 ms; dense → Prim 42 ms vs Kruskal 126 ms

0/1 knapsack DP

Which servers to switch on?

Each server is **on or off** (0/1), has a cost and a benefit, and the budget is the knapsack capacity. DP gives the **exact** best set.

`θ(n·W), W = budget in $0.01 steps`

Measured: grows linearly, ≈ 97 ns per table cell

Multiple-choice knapsack

How many copies?

For each service pick **exactly one option** — 1, 2, 3... copies — each with a cost and a latency saving, all within the VM budget.

`θ(W · Σ options)`

Budget $2.41 → $7.51/h cut latency 4.34 → 1.28 ms (−70%)

STAGES 6 & 7 · THE NP-HARD PART

Greedy placement + local search, checked against the exact optimum

Four greedy schedulers

- **Demand-first:** copies go where most users are
- **Marginal gain:** add the copy that cuts latency the most (CELF lazy updates)
- **Regret:** first serve the service that loses most if it misses its best city
- **FFD packing:** first-fit-decreasing bin packing — fits capacity first

Local search

Try **moving** one copy or **swapping** two; keep any change that lowers latency; repeat until nothing improves.

Distance from the proven optimum

Method	Greedy alone	+ Local search
Random placement	167 %	—
FFD packing	38.4 %	**≈0.5 %**
Demand-first	16.7 %	**≈0.5 %**
Regret	9.2 %	**≈0.5 %**
Marginal gain	8.7 %	**≈0.5 %**

Averaged over Waxman networks of 20–240 servers. Worst case after local search: about **3 %**.

When servers are nearly full (E5), only **FFD packing** always found a valid plan — so the solver uses it as a fallback.

DESIGN DECISIONS

Why we built it this way

Why not one exact DP for placement?

An exact DP must remember every server's remaining capacity, so its table grows with the **product of all capacities**. That is fine for about 8 servers but impossible at 25–150. We use DP where its tables stay small (budgets) and greedy + local search where they would explode.

Then how do we know greedy is good?

An exact solver (MILP) is used **only as a referee**: it proves the true optimum on smaller networks, and every greedy reports its gap. Result: about **0.5 %** from optimal. Every algorithm that builds the plan is our own code.

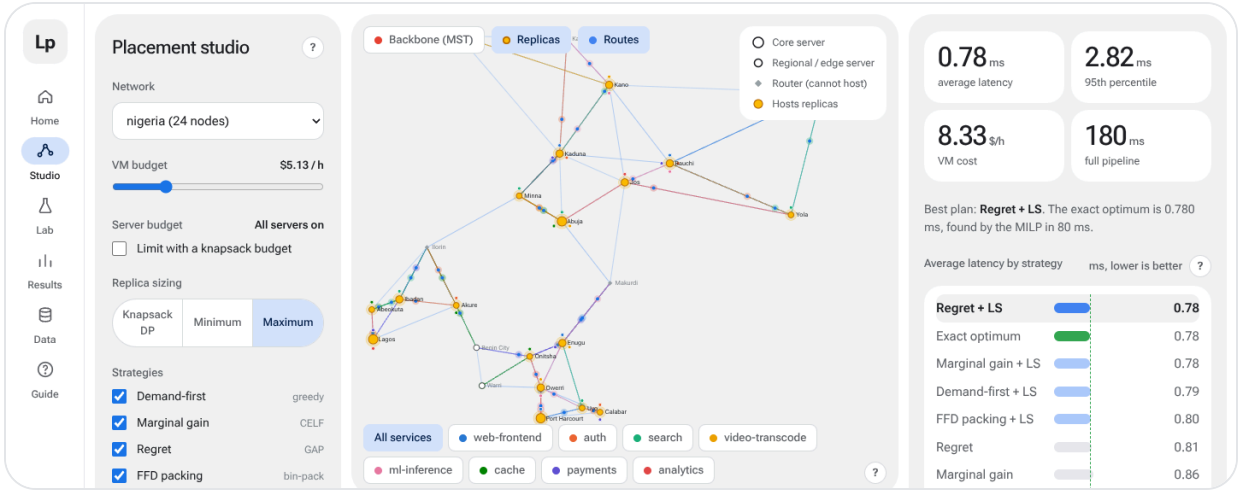
Every algorithm in the brief

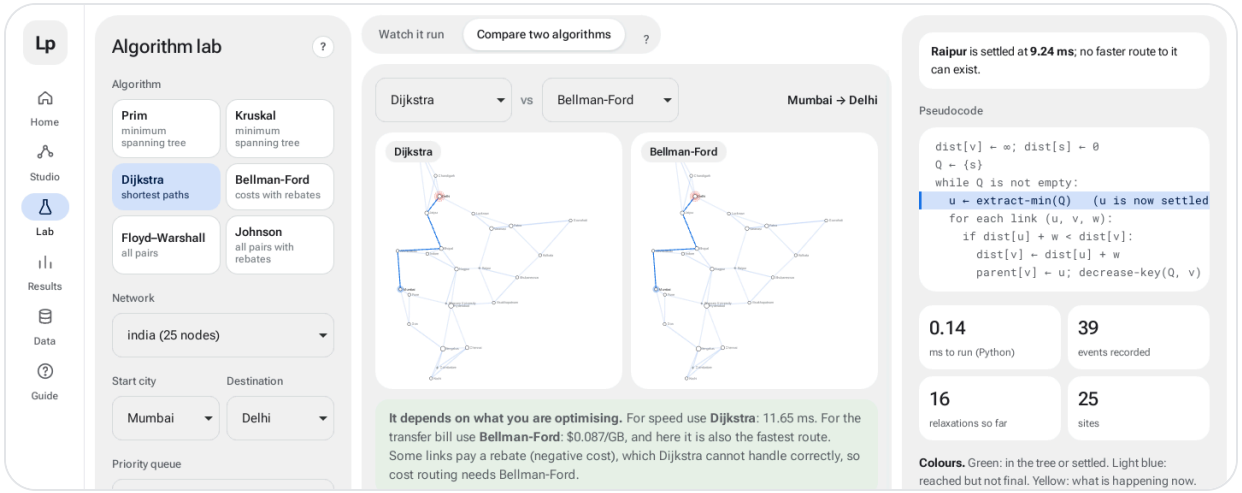
MST	**Prim, Kruskal** + union-find
Single source	**Dijkstra, Bellman-Ford**
All pairs	**Floyd–Warshall, Johnson**
Dynamic programming	**0/1 and multiple-choice knapsack**
Greedy	**4 schedulers** + local search, LPT batch
Priority queues	**binary, Fibonacci, pairing, lazy**

Each one has a real job: negative rebate costs are why Bellman-Ford and Johnson are needed, not just added for the list. Choices between alternatives are made automatically and **tested by experiments**.

THE PLATFORM

Everything runs in an interactive website





Home

Live map: requests travelling to their nearest copy.

Placement Studio

Pick a network and budget → **Solve** → plan, routes, advice.

Algorithm Lab

Watch any algorithm from a start to a destination, or **compare two**.

Results · Data · Guide

Experiment charts, downloads and a guided tour.

Python engine (NumPy, SciPy HiGHS) · FastAPI · React. Works offline too.

EXPERIMENTS · WHAT WE MEASURED

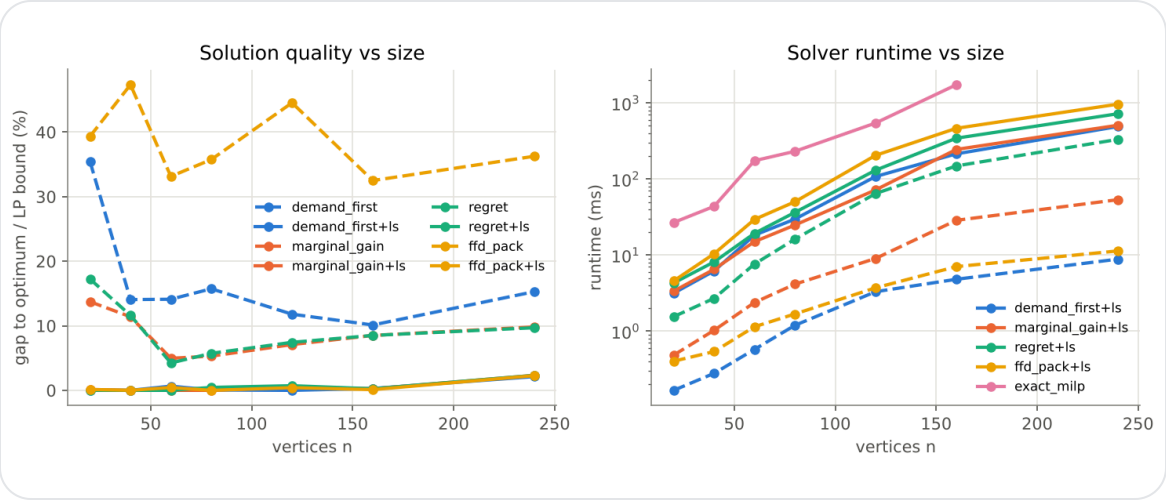
Eight experiments, one big lesson: measure, don't assume

2.33 ms
average latency, India network

−64%
vs a random placement

≈0.5%
from the proven optimum

191
automated tests, all passing



Fibonacci heap: best Big-O, but **slower** in practice (25.9 vs 17.7 ms) — pointer overhead beats the theory.

All pairs: NumPy Floyd–Warshall beat Johnson at n = 400 (277 vs 5015 ms dense).

MST: Kruskal wins sparse, Prim wins dense — exactly as theory predicts.

Budget & density: more money or more links → lower latency, with diminishing returns.

CONCLUSION

Right algorithm for each question → fast, near-optimal, provable plans

What we built

A complete system: 6 graph algorithms, 2 knapsack DPs, 4 greedy schedulers, 4 priority queues, an exact solver, 10 networks, 8 experiments and an interactive website.

What we found

Greedy + local search lands within **≈0.5 %** of the optimum, ≈5× faster than the exact solver. Placement cut latency by up to **64 %**.

What we learned

Big-O is a guide, not a verdict — constant factors and graph density decide the winner. Always check heuristics against an exact answer.

Thank you.

Questions? Live demo on the website.

Naman Singla · **Mohak Gupta**
Guided by Dr. Monday Jubrin Abdullahi
CSE RHINOS · Woxsen University